Лабораторна робота № 4

Мета роботи:

Сформувати навички інтеграції сенсорів у ROS2, реалізувати сенсорну ноду, організувати публікацію вимірювань у Topic та застосувати базову обробку (фільтрацію) даних.

Репозиторій з роботою: https://github.com/zakappaa/ROS2_Ginzburg

Хід роботи

Система складається з сенсорної ноди sensor_node, ноди керування рухом motion_node (з Лабораторної роботи №2) та симулятора turtlesim.

Створення пакету Sensor Node

У ~/ros2_ws/src створено пакет командою: ros2 pkg create --build-type ament_python --license Apache-2.0 --dependencies rclpy std_msgs --node-name sensor_node sensor_pkg (рис. 1). Робочий простір зібрано командою colcon build --packages-select sensor_pkg, коректність встановлення перевірено командами ros2 pkg list та ros2 pkg executables sensor_pkg (рис. 2).

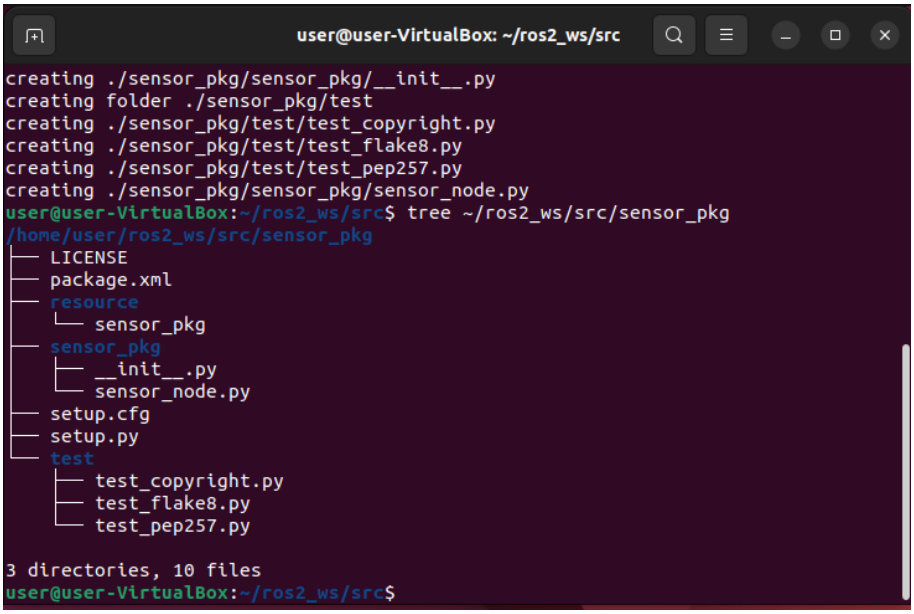


Рисунок 1 - Створення пакету та його структура

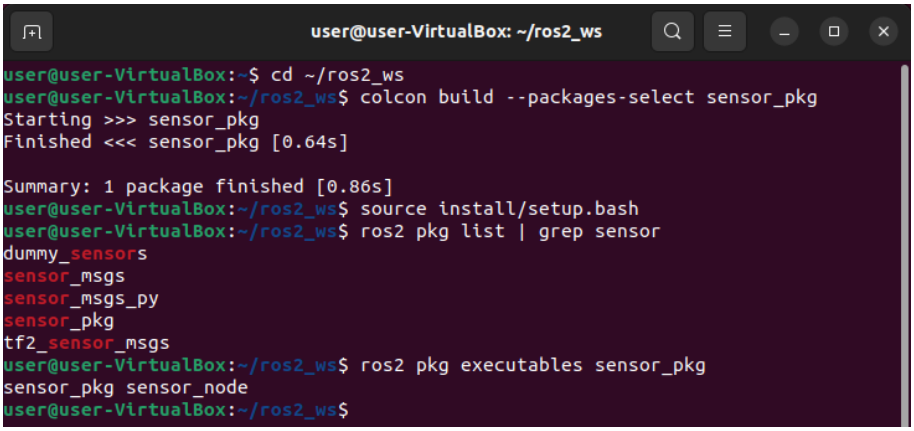


Рисунок 2 - Збірка workspace та перевірка встановлення пакету

Реалізація сенсорної ноди

Нода sensor_node реалізована класом SensorNode, успадкованим від rclpy.node.Node. Таймер з періодом 1 / publish_rate (0.1 с) викликає функцію timer_callback, яка виконує повний цикл обробки: зчитування => публікація сирих даних => усереднення => публікація відфільтрованих даних => порогова перевірка => публікація сигналу тривоги.

Ковзне вікно реалізовано структурою collections.deque з обмеженою довжиною (maxlen = window_size): при додаванні нового значення найстаріше автоматично видаляється. Порогова перевірка виконується по відфільтрованому значенню, тому одиничний викид не викликає хибної тривоги. Логи INFO та WARN обмежено однією появою на секунду, щоб не перевантажувати консоль при частоті 10 Гц.

Публікація даних

Ноду запущено командою ros2 run sensor_pkg sensor_node. Під час старту вона виводить параметри, далі раз на секунду показує поточні сирі та відфільтровані значення (рис. 3).

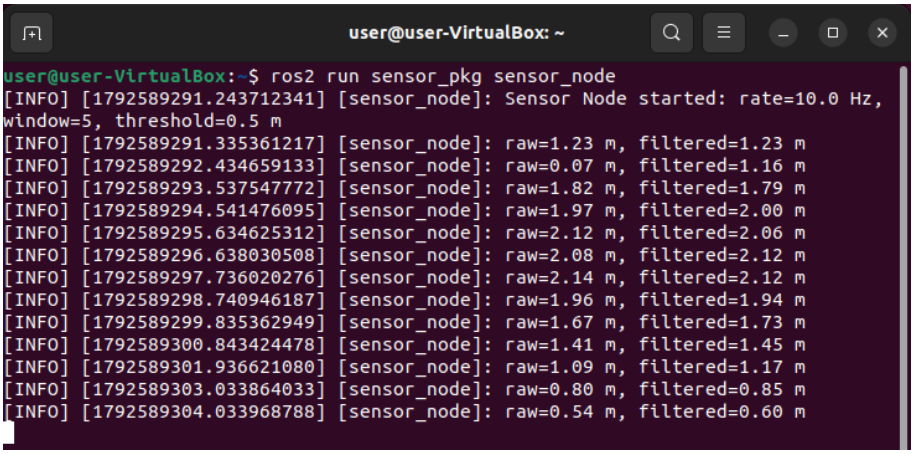


Рисунок 3 - Запуск сенсорної ноди та вивід у лог

Командами ros2 node list, ros2 topic list -t, ros2 topic info та ros2 topic hz перевірено наявність ноди й топіків та частоту публікації. Виміряна частота /sensor_data становить <значення average rate з рис. 4> Гц, що відповідає заданим 10 Гц (рис. 4).

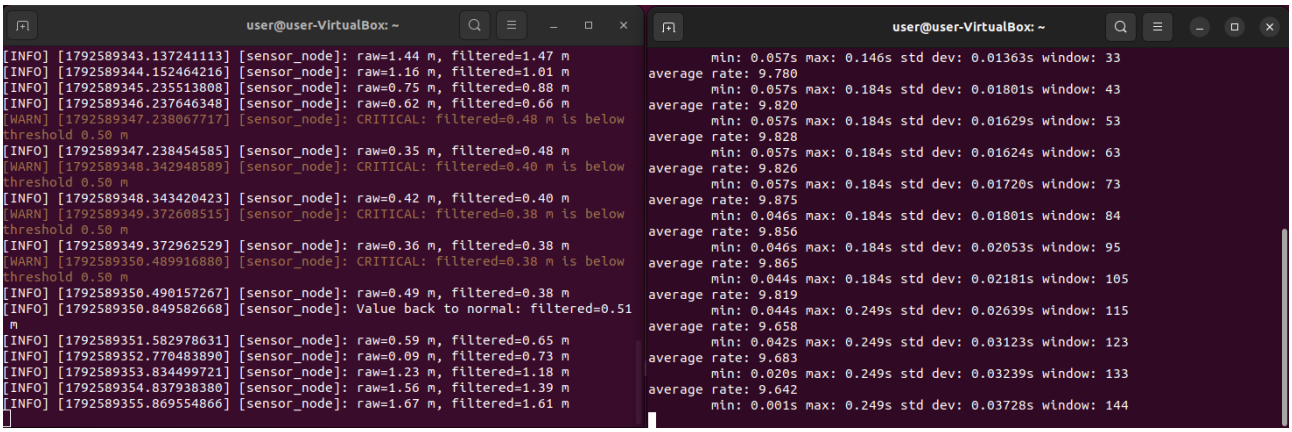


Рисунок 4 - Ноди, топіки та частота публікації /sensor_data

Отримання даних перевірено за допомогою ros2 topic echo: у двох терміналах одночасно виведено сирі (/sensor_data) та відфільтровані (/sensor_data_filtered) значення. Видно, що відфільтровані значення змінюються плавніше (рис. 5).

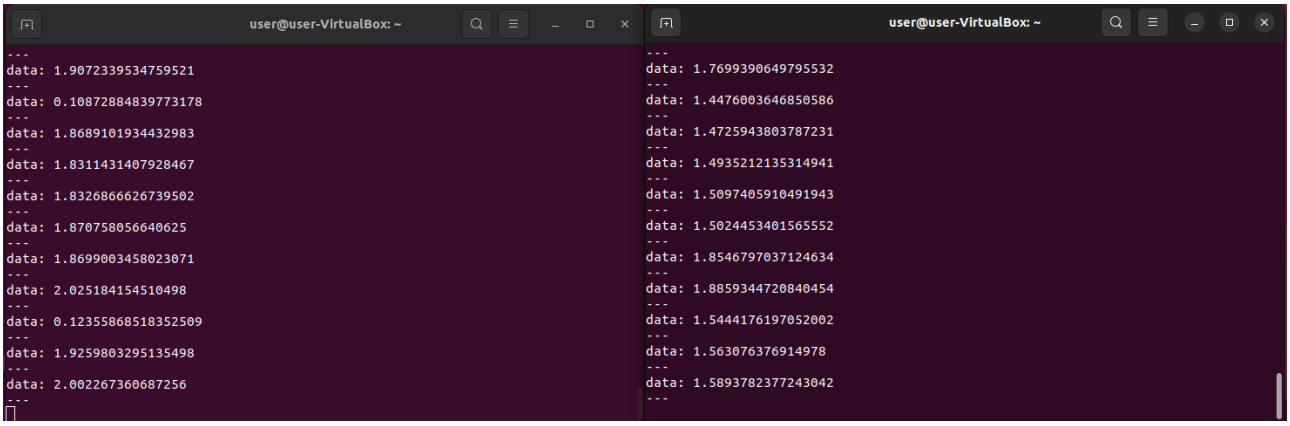


Рисунок 5 - Перегляд даних через ros2 topic echo: сирі та відфільтровані

Реакція системи на критичне значення

Коли відфільтрована відстань стає меншою за поріг 0.5 м, сенсорна нода виводить у журнал WARN повідомлення й публікує true у топік /sensor_alarm. Для демонстрації без очікування природного спуску порогове значення було змінено командою ros2 param set /sensor_node threshold 3.0, що негайно викликало тривогу (рис. 6). Після повернення порога до 0.5 м з'явилося повідомлення Value back to normal, а /sensor_alarm став false.

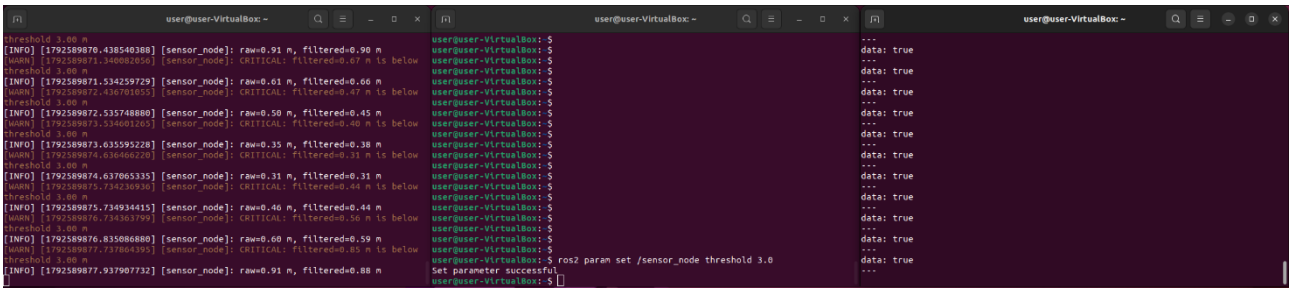


Рисунок 6 – WARN повідомлення сенсорної ноди та сигнал /sensor_alarm

Інтеграція: заборона руху при критичному значенні

Ноду керування рухом motion_node з Лабораторної роботи №2 доповнено підпискою на /sensor_alarm. Поки тривога активна, motion_node забороняє рух вперед (linear.x > 0 замінюється на 0), але дозволяє рух назад і поворот, щоб робот міг відійти від перешкоди. Заборона виводиться в журнал WARN повідомленням, а в топік статусу /motion_status публікується стан BLOCKED.

Для перевірки в /cmd_vel безперервно подавалася команда руху по колу (ros2 topic pub --rate 5). При тривозі сенсора черепаха припиняла рух уперед і

лише оберталася на місці, після зняття тривоги рух по колу відновлювався (рис. 7).

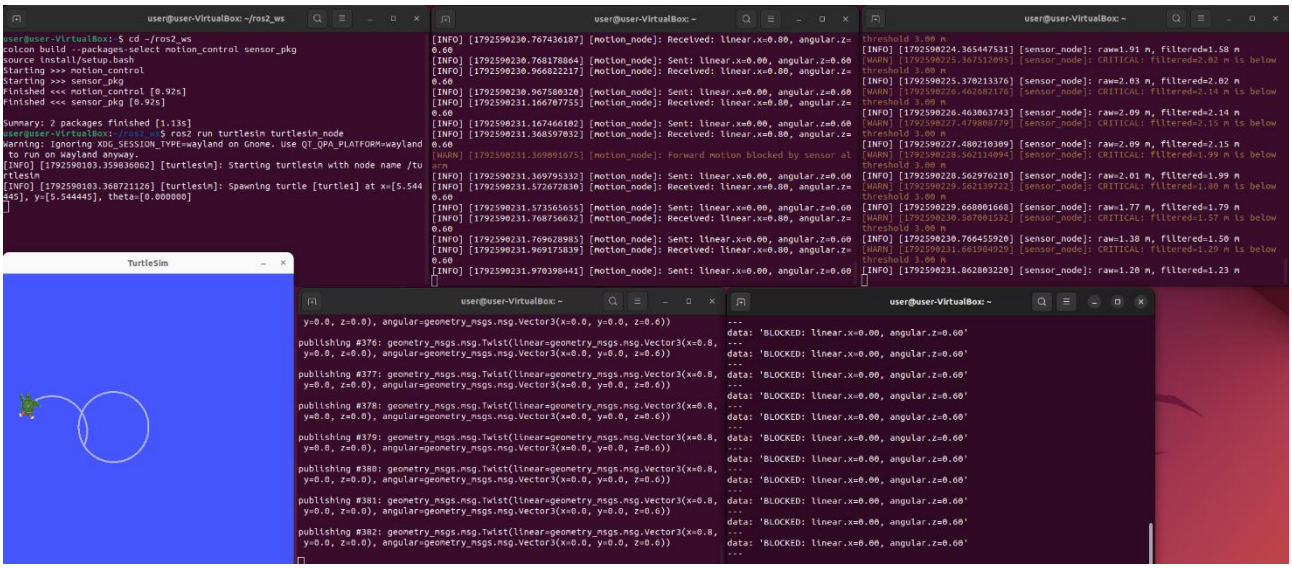


Рисунок 7 - Реакція системи: блокування руху вперед при тривозі сенсора

Висновок

У ході лабораторної роботи розроблено ноду sensor_node (sensor_pkg), яка імітує далекомір і публікує дані в топік /sensor_data (10 Гц, std_msgs/msg/Float32). Сигнал згладжено ковзним середнім (вікно 5) та передано в /sensor_data_filtered, а порогова перевірка формує WARN сповіщення та сигнал небезпеки в /sensor_alarm. Роботу ноди підтверджено за допомогою ros2 topic echo, ros2 topic hz та rqt_plot.

Ноду успішно інтегровано з motion_node: за сигналом тривоги рух уперед блокується з можливістю від'їзду назад. Це на практиці продемонструвало модульність ROS 2 - розширення функціоналу системи через топіки без переписування існуючих вузлів.